

Building an Internal AI Economics Practice

A practical guide for CTOs and engineering leaders to establish a rigorous, margin-aware discipline that treats software engineering as an economic system — and stops leaving money on the table.

FOR CTOS & ENGINEERING LEADERS

STRATEGIC FRAMEWORK

The Problem No One Is Naming

Most engineering organizations today are building AI features at a pace that far outstrips their ability to measure, understand, or control the economics of those features. Infrastructure bills climb. Token consumption balloons. Margin erodes. And the CFO and the architecture team have no shared language to even diagnose the problem together.

The gap is not technical. It is structural. Engineers optimize for performance and velocity. Finance optimizes for cost containment. Neither has a framework that bridges both worlds simultaneously — and that gap is costing organizations millions in preventable compute spend, misaligned incentives, and features that destroy margin at scale.

This guide introduces a discipline called **AI Economics** — a practice designed specifically for engineering leaders who need to close this gap. It is not an accounting exercise. It is a strategic operating system for how your team designs, ships, and manages AI-powered software in a world where infrastructure costs are variable, unpredictable, and structurally tied to user behavior.

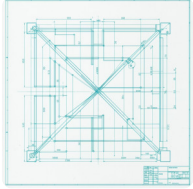


Without this discipline, even high-growth AI products can silently become margin destroyers. The warning signs are rarely visible until the damage is already done.



What Is an AI Economics Practice?

An AI Economics Practice is a formal discipline that treats software engineering as an economic system — with explicit inputs, outputs, margins, and feedback loops. It exists at the intersection of three organizational functions that rarely talk to each other: engineering architecture, product strategy, and financial operations.



Engineering Architecture

Defines how AI systems are structured, what models are called, at what frequency, and with what context windows. Every architectural decision has a direct cost signature.



Product Strategy

Determines which AI features get built, for whom, and at what usage tier. Product decisions drive consumption patterns that can make or break unit economics.



Financial Operations

Translates infrastructure spend into margin analysis, cohort-level cost attribution, and board-level reporting. Without this layer, engineering has no economic accountability.

The AI Economics Practice is the connective tissue between these three functions. It creates a shared language — a set of metrics, rituals, and decision-making frameworks — that allows a CTO and a CFO to sit in the same room, look at the same data, and make aligned decisions about which AI investments to accelerate and which to restructure.

An AI Economics Practice does not slow down engineering. It makes engineering investments legible — to the board, to the investor, and to the team itself.

Why You Need One: Two Existential Traps

The urgency of establishing this practice is not abstract. There are two specific failure modes — structural traps — that destroy margin in AI-native organizations. Most engineering leaders are already inside one of them without realizing it.

Trap 1: The Compute Reseller Trap

This occurs when your product's revenue model does not adequately price in the variable cost of AI inference. You become, in effect, a reseller of compute at a loss — buying tokens at full price and selling them bundled into a flat-rate subscription or per-seat license that was priced before you understood consumption at scale.

The trap tightens as your best customers — your power users — consume disproportionately more compute than your pricing model assumed. Revenue grows. Gross margin collapses. The faster you grow, the faster you bleed.

Trap 2: Infinite Power User Liability

Power users are not your best customers from a margin perspective. They are often your highest-liability customers. A single power user who sends 500 API calls per session can consume 100x the compute of an average user — at the same price point. This creates an unbounded liability that scales with your most engaged users.

Without an AI Economics Practice, there is no mechanism to detect this liability, model its trajectory, or restructure it before it becomes a crisis. By the time it appears in your financial statements, the architectural decisions that caused it are already baked into production.

-  Both traps are invisible until they are catastrophic. They do not appear in sprint velocity metrics, user satisfaction scores, or even early revenue growth. They appear in gross margin — and only when it is already too late to make easy adjustments.

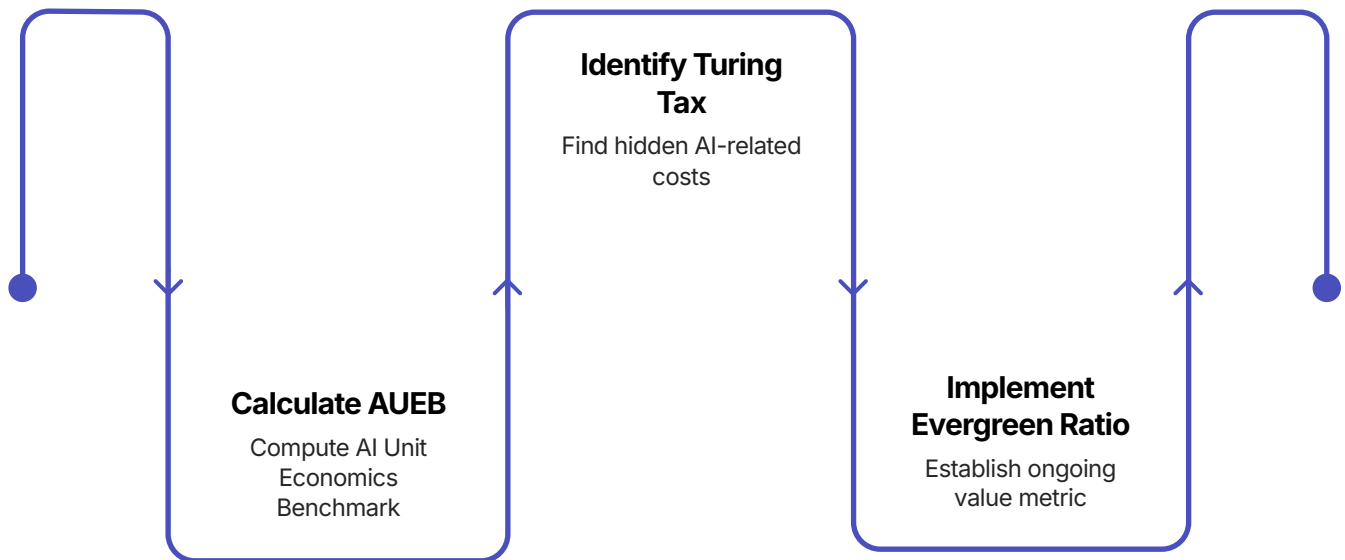
The Economic Stakes

To understand why this practice is board-level urgent, consider the margin mathematics at stake. A SaaS company with strong revenue growth and collapsing gross margin is not a growth company — it is a liability. AI-native products are particularly exposed because their cost structure is *consumption-driven*, not fixed.

60-80%	100x	3-5x
Target Gross Margin	Power User Consumption	Cost Growth vs. Revenue
Industry benchmark for healthy software businesses. AI-native products routinely fall below this without economic discipline.	The consumption differential between a power user and an average user in a poorly architected AI feature — at the same price point.	Infrastructure cost growth can outpace revenue growth by this multiple when compute costs are not actively managed at the feature level.

How to Start: The Three-Step Framework

Establishing an AI Economics Practice does not require a new team, a new tool, or a six-month transformation program. It requires three concrete actions, executed in sequence, that together create the foundation of an economically disciplined engineering organization.



Each step builds on the last. The AUEB gives you a baseline. The Turing Tax identifies your largest source of invisible drag. The Evergreen Ratio creates the forward-looking governance mechanism that prevents new features from introducing new liabilities. Together, they form a closed-loop economic operating system for your AI engineering practice.

Step 1: Calculate Your AUEB

AI Unit Economics Benchmark

What It Measures

The AUEB is your baseline cost-per-value-unit metric. It answers one question with precision:

What does it cost, in compute, to deliver one unit of AI-powered value to one user?

Defining "value unit" is the first creative act of your economics practice. For a code assistant, it might be a accepted suggestion. For a summarization tool, it might be a document processed. For a chat interface, it might be a resolved intent. The definition must be tied to an outcome the user cares about — not an internal system event.

Once defined, the AUEB becomes your most important engineering metric. It replaces vague conversations about "compute costs" with a precise, trackable, improvable number that every engineering decision can be evaluated against.

How to Calculate It

1. **Define your value unit** — the atomic user outcome your AI feature delivers
2. **Instrument your infrastructure** — capture token counts, model calls, and latency per user session, mapped to value unit delivery
3. **Calculate fully-loaded cost** — include inference cost, embedding cost, retrieval cost, and any orchestration overhead
4. **Divide by value units delivered** — this is your AUEB baseline
5. **Segment by user cohort** — your AUEB will vary dramatically between user segments; this variation is your first diagnostic signal

 Start with a single feature, not your entire platform. A focused AUEB on your highest-cost feature will generate more insight in two weeks than a broad audit will in two months.

Step 2: Identify Your Turing Tax



The **Turing Tax** is the hidden cost of intelligence — the compute overhead your system pays not to produce value, but simply to *appear* intelligent. It includes every token spent on context padding, every model call made defensively rather than purposefully, every re-ranking operation run by default rather than by necessity.

Most AI systems carry a Turing Tax of 30–60% of their total compute spend. That means for every dollar spent on inference, only 40–70 cents is doing genuine value-generating work. The rest is paying for the appearance of sophistication — longer prompts, redundant retrievals, over-engineered context windows designed to hedge against edge cases that rarely materialize.

Identifying your Turing Tax is an audit exercise. You are looking for three categories of overhead:

Defensive Compute

Model calls, retrieval steps, or context inclusions that exist to handle edge cases, but fire on every request regardless of whether the edge case applies.

Vanity Intelligence

Features that consume significant compute but produce marginal improvement in user-perceived quality. Often the result of benchmarking against model capability rather than user outcomes.

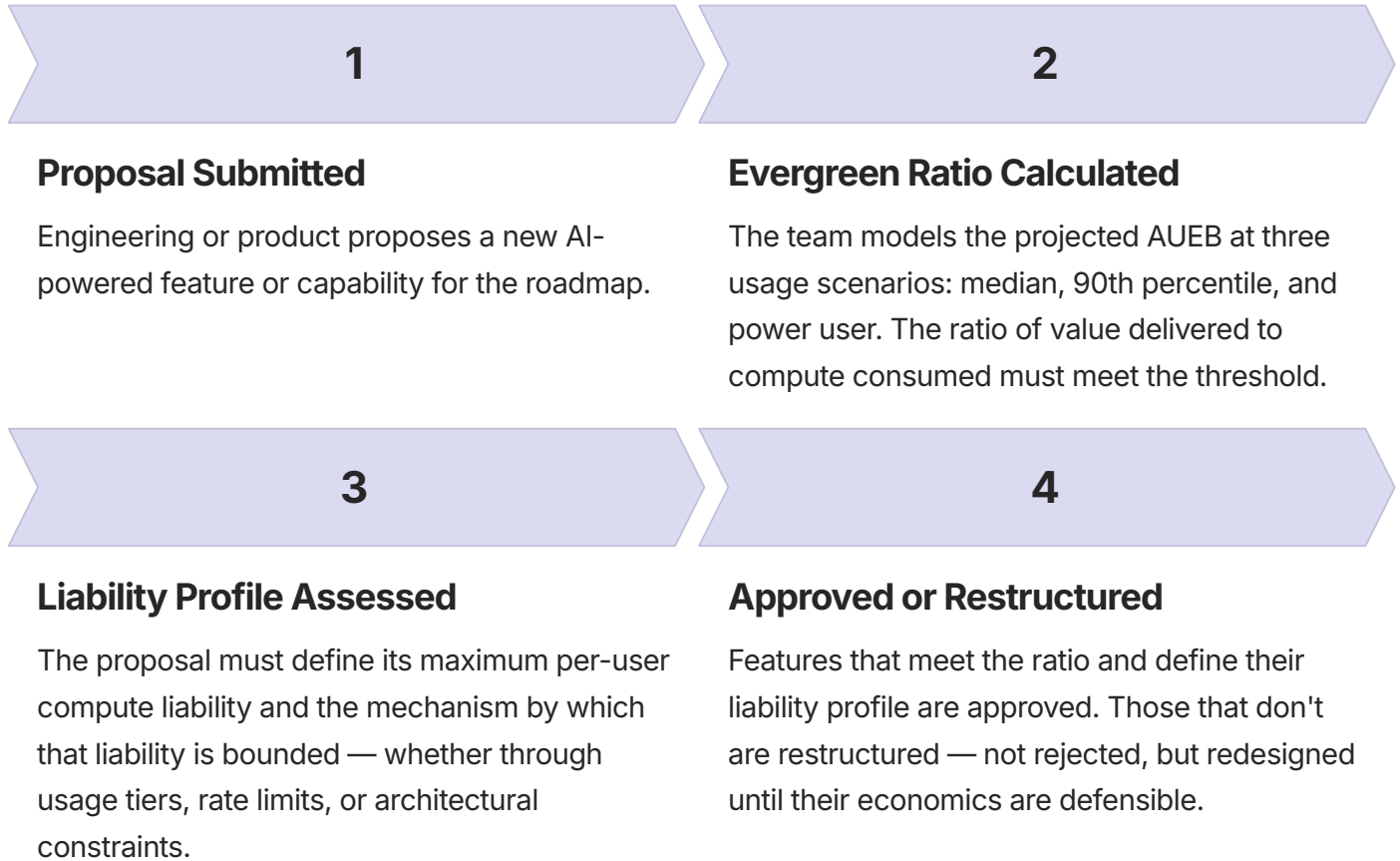
Architectural Debt Tax

Compute overhead introduced by early architectural decisions that were never revisited as usage scaled. Often the largest single component of the Turing Tax in mature AI products.

Step 3: Implement the Evergreen Ratio

The Forward-Looking Governance Mechanism

The Evergreen Ratio is the discipline that prevents your economics practice from becoming a retrospective audit exercise. It is a forward-looking governance gate applied to every new AI feature proposal before it enters the engineering roadmap.



The Evergreen Ratio shifts the cultural center of gravity in your engineering organization. It makes economics a first-class design constraint — as fundamental as performance, security, or reliability. Over time, engineers internalize the framework and begin applying it instinctively, without the formal gate. That internalization is the signal that your AI Economics Practice has become a genuine organizational capability.

The Evergreen Ratio is not a finance review. It is an engineering standard. The goal is not to slow features down — it is to ship features that remain economically healthy as they scale.

Take the Next Step

Learn From the Founder of the Framework

The AI Economics framework — the AUEB, the Turing Tax, the Evergreen Ratio — is not theoretical. It has been developed and refined through direct engagement with engineering organizations navigating the real margin pressures of building AI-native products at scale. The fastest way to establish this discipline inside your organization is not to reverse-engineer it from first principles. It is to learn it from the source.

What You Will Leave With

- A fully calculated AUEB for your highest-cost AI feature
- A complete Turing Tax audit methodology your team can run independently
- An Evergreen Ratio template ready to integrate into your existing roadmap process
- A board-ready AI Economics narrative for your next investor or executive review

Where to Enroll

Track 24: AI Economics & Margin Engineering

Designed exclusively for CTOs and senior engineering leaders. Instructional, hands-on, and immediately applicable to your current architecture and cost structure.

Led by Richard Ewing, the founder of the AI Economics framework.

[Enroll at richardewing.io](https://richardewing.io)

[Learn More](#)

- ✓ The engineering organizations that build this discipline now will have a structural margin advantage over those that retrofit it later. The window to act before this becomes table stakes is closing.